

COMICCRAFT

AI Comic Story Creator using Gemini Models

PROJECT DEMONSTRATION PHASE

This phase demonstrates the working application from user input to AI-generated comic preview and final PDF export.

1. Demonstration Phase Overview

The demonstration phase presents the complete end-to-end working flow of ComicCraft. The application accepts a user's story prompt, character name, setting, story tone, and art style. The backend then coordinates Gemini Flash for the structured comic outline, Gemini Pro for detailed narration and dialogue, and Stable Diffusion for comic-style illustrations. The generated panels are displayed in a web preview and can be exported as a downloadable PDF.

Demonstration objective: To show that ComicCraft can transform a simple creative idea into a complete five-panel comic with text, illustrations, preview, and PDF export through a single web-based workflow.

2. Demonstration Environment

Component	Used in Demonstration
Frontend	HTML, CSS and Jinja2 templates
Backend	FastAPI with Uvicorn
Story outline	Gemini Flash
Narration and dialogue	Gemini Pro
Image generation	Stable Diffusion through Hugging Face Diffusers
PDF export	FPDF
Local application	http://127.0.0.1:8000
API testing	http://127.0.0.1:8000/docs

3. Demonstration Flow

Step 1 - Start the application

Launch the FastAPI server using Uvicorn and open the ComicCraft homepage in a browser.

Step 2 - Enter comic requirements

Provide the story prompt, main character name, setting, story tone, and preferred art style.

Step 3 - Submit the request

Submit the form. FastAPI receives and processes the user's input.

Step 4 - Generate the outline

Gemini Flash creates a structured five-panel outline containing panel number, title, scene description, and image-generation prompt.

Step 5 - Generate story content

Gemini Pro expands the outline into narration and character dialogue for the panels.

Step 6 - Generate illustrations

Stable Diffusion generates a comic-style illustration for each panel using the corresponding image prompt.

Step 7 - Build the comic layout

The application matches each generated image with its panel story and organizes the complete layout.

Step 8 - Display the preview

Jinja2 renders the generated comic panel-by-panel on the Comic Preview page.

Step 9 - Export the comic

The user selects the PDF download option. FPDF compiles the panel images and narrative content into a multi-page PDF.

Step 10 - Confirm completion

The user is redirected to the export-success page, confirming that the comic was successfully created and downloaded.

4. Sample Demonstration Scenario

A suitable demonstration can use the documented example: “A brave fox exploring an enchanted forest.” The demonstrator can enter the following values to show personalization:

Input field	Sample value
Story Prompt	A brave fox exploring an enchanted forest
Main Character Name	Fox
Setting	Enchanted forest
Story Tone	Dramatic
Art Style	Anime

5. Homepage Demonstration

The homepage is the starting point of the demonstration. It provides a structured form where the user enters the story prompt, character name, setting, tone, and art style. The documented interface is designed to be simple and user-friendly so that users can create a comic without requiring technical expertise.

- Enter the main story idea in the Story Prompt field.
- Enter the name of the main character.
- Specify the setting, such as forest, school, city, or space.
- Choose a story tone such as dramatic, funny, poetic, or light-hearted.
- Choose an art style such as anime, pixel art, comic book, or realistic.
- Submit the form to begin comic generation.

6. AI Generation Demonstration

After form submission, the backend coordinates the AI pipeline. The documented workflow calls `generate_outline()` from `gemini_flash.py`, `generate_story()` from `gemini_pro.py`, and `generate_image()` from `image_generator.py`. The generated content is then organized by `build_comic_layout()` before it is passed to the frontend for display.

Stage	Function / Module	Demonstration result
1	<code>generate_outline()</code> / <code>gemini_flash.py</code>	Structured five-panel outline
2	<code>generate_story()</code> / <code>gemini_pro.py</code>	Narration and character dialogue
3	<code>generate_image()</code> / <code>image_generator.py</code>	Comic-style image for each panel
4	<code>build_comic_layout()</code> / <code>layout_builder.py</code>	Panel images matched with story text
5	<code>save_pdf()</code> / <code>exporters.py</code>	Multi-page downloadable comic PDF

7. Comic Preview Demonstration

The Comic Preview page presents the generated comic sequentially. Each panel contains a panel title, AI-generated illustration, scene description, caption, narration, and the image-prompt reference. This allows the demonstrator to review the complete story before exporting it.

- Verify that all generated panels appear in sequence.
- Check that each panel contains its corresponding image and text.
- Show how the scene description and narration complement the illustration.
- Explain that the preview stage allows the user to review the generated comic before export.

8. PDF Export Demonstration

On the Comic Preview page, the user selects “Download Your Comic as PDF.” The application uses `save_pdf()` in `exporters.py` and the `FPDF` library to compile the comic into a multi-page PDF. The exported document contains the generated panel images and narrative content and is stored in the `static/exports` folder according to the project documentation.

Expected demonstration result: A complete comic PDF is generated and downloaded to the user's device.

9. Export Success Demonstration

After the export is triggered, the application displays the Comic Export Success page. The page confirms that the AI-powered comic has been created and downloaded and provides a “Go Create Another Comic” action to return to the homepage and begin a new comic.

10. API Demonstration

For a technical demonstration, the FastAPI interactive documentation can be opened at <http://127.0.0.1:8000/docs>. The project documents API routes including `/generate-comic/json`, `/generate`, `/export-success`, and `/test-image`. The JSON generation route accepts a structured request and returns layout data and the generated PDF path, while `/test-image` can be used to demonstrate image generation separately.

11. Demonstration Checklist

✓	FastAPI server starts successfully.
✓	ComicCraft homepage loads in the browser.
✓	All five user inputs can be entered.
✓	Comic generation request is submitted successfully.
✓	Five-panel outline is generated.
✓	Narration and dialogue are generated.
✓	Illustrations are generated for the panels.
✓	Comic Preview displays the complete generated comic.
✓	PDF export is triggered successfully.
✓	Export Success page is displayed.

12. Demonstration Conclusion

The demonstration shows the complete ComicCraft workflow: personalized input collection, AI-based outline generation, story and dialogue creation, illustration generation, panel layout,

browser-based preview, and PDF export. The project documentation describes this as an end-to-end experience that transforms a simple prompt into a visually engaging comic through a web interface.

Note: This demonstration phase is derived from the uploaded ComicCraft project documentation. It describes the documented workflow and expected application behavior; it does not claim that a live deployment was executed during preparation of this report.